Diaconis-Ylvisaker logistic regression in brglm2

Federico Boiocchi*

*Visiting student, Department of Statistics, University of Warwick

September 19, 2025

Abstract

In this work, we detail the implementation of the Maximum Diaconis-Ylvisaker prior penalized likelihood for logistic regression in the R package brglm2. The main reference has been the article (Sterzinger and Kosmidis, 2023) and the related Julia code detailed in `https://github.com/psterzinger/MDYPL`. The primary aim of the project was to program exclusively in R the solver for the Approximate message passing state evolution system, whose solution characterizes the asymptotic behaviour of the Diaconis-Ylvisaker estimator for logistic regression. The main output of the project consists of three functions `se1()`, `se0()` and `solve_se()`, that respectively approximate the system with and without the intercept and solve it.

1 Background

1.1 Logistic regression framework

Logistic regression is one of the most widely used GLM for predicting binary outcomes or inferring covariates effects. For each statistical unit u_i with $i = 1, \dots, n$, the model associates a binary response variable y_i to a vector of explanatory variables through a transformed linear predictor. The idea is to model the conditional expected value of the response as follows:

$$\mathbb{E}[y_i|\boldsymbol{x}_i] = \frac{\exp(\mathbf{x}_i^\top \boldsymbol{\beta})}{1 + \exp(\mathbf{x}_i^\top \boldsymbol{\beta})} \quad \text{with } y_i \sim \text{Ber}(\mu_i). \tag{1}$$

Since the mean of a Bernoulli random variable is its success probability, we are in practice modeling $\mathbb{P}(y_i = 1|\mathbf{x}_i)$ using an inverse logit transformation of $\mathbf{x}_i^\top \boldsymbol{\beta}$. Therefore, the regression problem boils down to finding the unknown regression coefficients $(\beta_0, \beta_1, \dots, \beta_p)$ given both the vector of responses $\mathbf{y}$ and the design matrix of covariates $\mathbf{X}$. The standard way of doing so is using a maximum likelihood (ML) estimation approach, namely, $\boldsymbol{\beta}$ is estimated through $\hat{\boldsymbol{\beta}}_{\text{ML}}$, where:

$$\hat{\boldsymbol{\beta}}_{\text{ML}} = \underset{\boldsymbol{\beta} \in \mathbb{R}^p}{\text{argmax}} \ \ln(\text{L}(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X})),$$

with $\text{L}(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X})$ the likelihood of a sample of Bernoulli random variables whose means are modeled as in (1); The explicit definition of the log-likelihood We are dealing with is the following:

$$\ln(\text{L}(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X})) = \sum_{i=1}^{n} \{y_i \ \mathbf{x}_i^\top \boldsymbol{\beta} - \ln(1 + \exp(\mathbf{x}_i^\top \boldsymbol{\beta}))\} \tag{2}$$

A first remark is that this maximization problem cannot be solved analytically, but instead has to be solved numerically via a variant of the Newton-Raphson algorithm called Fisher scoring:

$$\boldsymbol{\beta}_{k+1} = \boldsymbol{\beta}_k - \text{I}(\boldsymbol{\beta}_k)^{-1} \text{S}(\boldsymbol{\beta}_k). \tag{3}$$

This is an algorithm that produces iterates that converge to the correct solution. Since we want to find the maximizer of the log-likelihood function $l(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X})$, we can reinterpret the same problem as solving the following nonlinear system:

$$\nabla l(\boldsymbol{\beta}_k) = \mathbf{0}, \tag{4}$$

which is the usual first-order optimality condition used to solve optimization problems. In this context, the gradient of the log-likelihood is often called the score function and is denoted with $\mathrm{S}(\boldsymbol{\beta}_k)$. Consequently, in (3), the score function is simply describing the nonlinear system to be solved, while the Fisher information matrix represents the Jacobian of the system. Since the Jacobian of the gradient is the Hessian matrix, the observed Fisher information matrix is the Hessian of the log-likelihood:

$$\mathrm{I}(\boldsymbol{\beta}) = \mathbf{H}_{l(\boldsymbol{\beta},\mathbf{y},\mathbf{X})}.$$

An essential remark is that the numerical maximization of the log-likelihood does not solve the problem of estimating $\boldsymbol{\beta}$ in every setting of $\{\mathbf{y}, \mathbf{X}\}$. In this regard, two distinct problems could arise, namely: data separability and high-dimensionality. The first problem, loosely speaking, involves the existence of a hyperplane in the variables space that perfectly separates the statistical units according to their binary responses; if this happens $\hat{\boldsymbol{\beta}}_{\mathrm{ML}}$ does not exist (at least one of its components is not finite) as shown in (Albert and Anderson, 1984). Another issue, detailed in (Sur and Candes, 2019) concerns the existence of a phase transition curve in the plane of the aspect ratio $k = \frac{p}{n}$ vs asymptotic standard deviation of the linear predictor $\sqrt{\mathrm{Var}(\mathbf{x}_i^\top \boldsymbol{\beta})} \to \gamma$. In this (k, γ) plane, $\hat{\boldsymbol{\beta}}_{\mathrm{ML}}$ does not exist with probability one for the combination of (k, γ) values above the transition curve (blue area); on the contrary, the ML estimate exists with probability one for (k, γ) below the curve (yellow region). Figure 1 on the left represents the phase transition curve simulated via montecarlo using R, while on the right it is represented the separability problem.

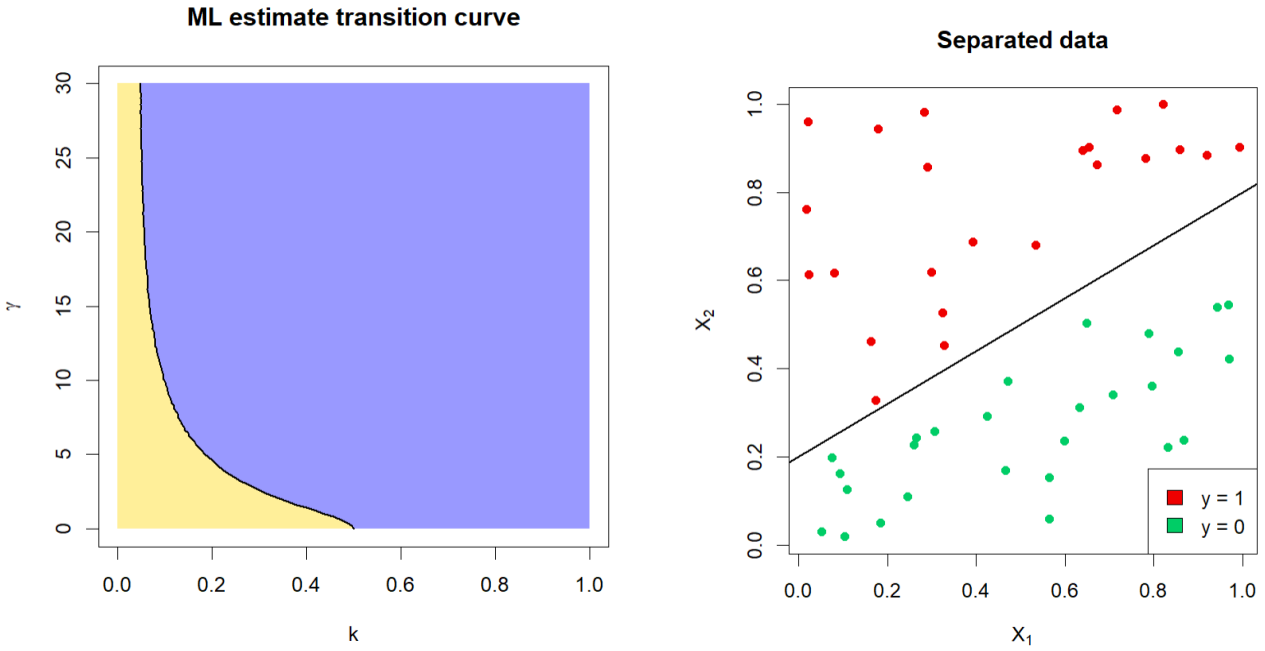

Figure 1: On the left: Phase transition curve approximated via Montecarlo. On the right: Data separability with two explanatory variables

To overcome these issues, several maximum penalized likelihood methods have been proposed. The idea of these approaches is to find the maximizer of a penalized version of the log-likelihood:

$$\hat{\boldsymbol{\beta}}_{\mathrm{MPL}} = \underset{\boldsymbol{\beta}\in\mathbb{R}^p}{\operatorname{argmax}} \ln(\mathrm{L}(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X})) + \ln(\pi(\boldsymbol{\beta})). \tag{5}$$

This penalization regularizes the problem and, if well chosen, solves the separability issue and extends the existence of the estimator of the regression coefficients to all the points in the (k, γ) plane. Another advantage of penalized regression is its Bayesian interpretation as a Maximum a posteriori (MAP) estimator. This follows from interpreting the maximizer of (5) as the mode of the posterior distribution in a Bayesian model:

$$\hat{\boldsymbol{\theta}}_{\mathrm{MAP}} = \underset{\theta\in\Theta}{\operatorname{argmax}}\ \pi(\boldsymbol{\theta}|\mathbf{y}, \mathbf{X}) = \underset{\theta\in\Theta}{\operatorname{argmax}} \frac{\mathrm{L}(\mathbf{y}, \mathbf{X}, \boldsymbol{\theta})\pi(\boldsymbol{\theta})}{m(\mathbf{y}, \mathbf{X})} = \underset{\theta\in\Theta}{\operatorname{argmax}}\ \ln(\mathbf{y}, \mathbf{X}, \boldsymbol{\theta}) + \ln(\boldsymbol{\theta}).$$

1.2 Diaconis-Ylvisaker penalized logistic regression

In our case, we specify a Diaconis-Ylvisaker (DY) prior on the regression coefficients of our logistic regression model; the DY prior has the following form

$$\boldsymbol{\beta} \sim \text{DY}(\alpha, \boldsymbol{\beta}_P) \quad \text{with} \quad f(\boldsymbol{\beta}) = \exp\left(\frac{1-\alpha}{\alpha} \sum_{i=1}^{n} \{ g^{-1}(\mathbf{x}_i^\top \boldsymbol{\beta}_P)\ \mathbf{x}_i^\top \boldsymbol{\beta} - \ln(1+\exp(\mathbf{x}_i^\top \boldsymbol{\beta})) \} + C \right),$$

where α is a hyperparameter controlling the variability of the prior while $\boldsymbol{\beta}_P$ is its mode. C is just a normalizing constant. Given the definition of the prior, we are now able to define the DY prior penalized log likelihood:

$$l^*(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X}) = \frac{1}{\alpha} \sum_{i=1}^{n} \{ (\alpha y_i + (1-\alpha) g^{-1}(\mathbf{x}_i^\top \boldsymbol{\beta}_P))\ \mathbf{x}_i^\top \boldsymbol{\beta} - \ln(1+\exp(\mathbf{x}_i^\top \boldsymbol{\beta})) \}. \tag{6}$$

Consequently, the Diaconis-Ylvisaker penalized estimator $\hat{\boldsymbol{\beta}}_{\text{DY}}$is simply the maximizer of $l^*(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X})$. As can be observed in (6), the penalized log-likelihood is exactly the standard log-likelihood detailed in (2) but computed on modified responses $y_i^* = \alpha y_i + (1-\alpha) g^{-1}(\mathbf{x}_i^\top \boldsymbol{\beta})$. Therefore, we have that:

$$l^*(\boldsymbol{\beta}, \mathbf{y}, \mathbf{X}) = \frac{1}{\alpha} l(\boldsymbol{\beta}, \mathbf{y}^*, \mathbf{X}).$$

Thanks to this result $\hat{\boldsymbol{\beta}}_{\text{DY}}$ is directly computable using Fisher-scoring on modified responses $\mathbf{y}^*$. For this reason, given the simplicity of the estimation procedure, this report will not focus on implementing the DY penalized logistic regression fitter in R, but rather on the theory and code used to describe the aggregate asymptotic behavior of $\hat{\boldsymbol{\beta}}_{\text{DY}}$, which is way more complicated. A last remark is that, depending on the choice of the value α we are able to perform shrinkage of the coefficients estimates towards the mode of the DY prior; this is particularly evident if we look at the expression in (6). Indeed, as $\alpha \to 0$ the log likelihood becomes the DY prior and its argmax becomes the mode, on the contrary, when $\alpha \to 1$ we recover the ML estimate.

2 Asymptotic behaviour of $\hat{\boldsymbol{\beta}}_{\text{DY}}$

The existence of $\hat{\boldsymbol{\beta}}_{\text{DY}}$ across all the (k, γ) plane makes it a very appealing estimator in the proportional asymptotics setting where $p/n \to k \in (0, 1)$. Since the DY estimator is defined as a ML estimator on transformed responses, its asymptotic properties are well-understood in a fixed p, $n \to \infty$ setting.

$$\hat{\boldsymbol{\beta}}_{\text{DY}} \underset{n\to\infty}{\overset{\text{A}}{\sim}} N\left(\boldsymbol{\beta}, \frac{1}{n} \text{I}(\boldsymbol{\beta})^{-1} \right)$$

However, all the classical properties of Maximum Likelihood estimators, such as asymptotic normality and Cramer-Rao efficiency, no longer hold in the proportional asymptotic setting ($p/n \to k \in (0, 1)$). This is a huge issue because without asymptotic distributional results on the regression coefficients, we cannot estimate confidence intervals, p-values, standard deviations, or compare models. Therefore, we need a theory to explain the proportional asymptotic behavior of $\hat{\boldsymbol{\beta}}_{\text{DY}}$. This is the primary focus of the article (Sterzinger and Kosmidis, 2023), which develops a Generalized Approximate Message Passing (GAMP) algorithm suited for DY logistic regression in order to describe the aggregate proportional asymptotic behaviour of $\hat{\boldsymbol{\beta}}_{\text{DY}}$. The idea is to construct an AMP recursion to estimate $\boldsymbol{\beta}$, namely an iterative algorithm that leads to the same solution of the Fisher scoring, but in such a way that each component of each iteration $\boldsymbol{\beta}_t$ for $p/n \to k \in (0, 1)$, has approximately the same empirical distribution of $f_t(\mu_t \beta + \sigma_t \xi)$. In this case, f_t is an arbitrary function that influences the AMP recursion, $\xi \sim N(0, 1)$ is a white noise, β is the unknown true signal, and μ_t and σ_t are parameters that can be found by solving an auxiliary state evolution system. Depending on the choice of f_t and another function g_t various AMP recursions suited for different models can be defined in order to obtain proportional asymptotic results on the distribution of the resulting iterates. Consequently, solving this nonlinear system allows us to determine the proportional asymptotic distribution of $\hat{\boldsymbol{\beta}}_{\text{DY}}$. Moreover, since we are able to describe asymptotically the distribution of the iterates, we can also define the aggregate asymptotic error we are committing at each iteration, in this regard the following results holds:

$$\sup_{\psi \in \text{PL}_2(r,1)} \left| \frac{1}{p} \sum_{j=1}^{p} \psi(\beta_j^{k+1}, \beta_j) - \mathbb{E}(\psi(\mu_{k+1}\overline{\beta} + \sigma_{k+1} G_{k+1}, \overline{\beta})) \right| \overset{p/n \to \delta \in (0,1)}{\longrightarrow} 0 \tag{7}$$

where PL_2 is the set of all Pseudo-Lipschitz functions from $\mathbb{R}^2$ to $\mathbb{R}$. In this report we will shorten the details about the AMP theory in order to focus on the approximation and solution of the state evolution system and not on its derivation. However, a detailed overview of approximate message passing algorithms applied to different models is given in (Feng and Samsworth, 2022). The expression in (7), when adapted to the DY logistic regression framework, allows us to produce an AMP "convergence machinery", namely a theorem that, depending on the function we select, describes the aggregate asymptotic behavior of $\hat{\boldsymbol{\beta}}_{\text{DY}}$:

$$\frac{1}{p}\sum_{j=1}^{p}\psi(\hat{\beta}_j^{\text{DY}} - \mu_*\beta_j, \beta_j) \overset{p/n\to\delta\in(0,1)}{\longrightarrow} \mathbb{E}[\psi(\sigma_* G, \overline{\beta})]. \tag{8}$$

This result, presented as Theorem 3.1 in (Sterzinger and Kosmidis, 2023), allows us to characterize the aggregate bias, aggregate variance, and aggregate MSE of $\hat{\boldsymbol{\beta}}_{\text{DY}}$. It is now evident how important it is to find the parameters (μ_*, b_*, σ_*) which are the solution of the AMP state evolution system. In this regard, in (8), different choices of $\psi(t,u)$ allow us to recover various asymptotic properties of $\hat{\boldsymbol{\beta}}_{\text{DY}}$. One of the most relevant results is stated below:

$$\frac{1}{p}\sum_{j=1}^{p}\psi\left(\frac{1}{\mu_*}\hat{\beta}_j^{\text{DY}} - \beta_j\right) \overset{\text{a.s.}}{\longrightarrow} 0, \quad \text{for } \psi(t,u) = \frac{t}{\mu_*}, \tag{9}$$

namely, the DY estimator, when adjusted with component-wise rescaling $1/\mu_*$, exhibits an asymptotic aggregate bias that converges to zero. Another important result adapted to $\hat{\boldsymbol{\beta}}_{\text{DY}}$ but fundamentally inspired by Theorem 4 in (Sur and Candes, 2019) describes the distribution of the Likelihood ratio statistics Λ_I in the high-dimensional logistic regression framework:

$$2\Lambda_I \overset{d}{\longrightarrow} \frac{k\sigma_*^2}{b_*}\chi_k^2, \tag{10}$$

where k is the number of degrees of freedom, namely the difference between the number of features in the full model and the nested one. As we can observe, this scaled version of Wilk's theorem heavily depends on the solution (μ_*, b_*, σ_*) of the state evolution system.

3 State evolution system

In the case of a logistic regression model without intercept in the linear predictor, the state evolution system whose solution (μ_*, b_*, σ_*) governs the asymptotic behavior of $\hat{\boldsymbol{\beta}}_{\text{DY}}$ is given by the following function $\vec{\text{F}}(\mu, b, \sigma) : \mathbb{R}^3 \to \mathbb{R}^3$ set to be equal to $\mathbf{0}$

$$\mathbb{E}\left[2g^{-1}(Z)Z\left\{\frac{1+\alpha}{2} - g^{-1}\left(\text{prox}_b\left(Z_* + \frac{1+\alpha}{2}b\right)\right)\right\}\right] = 0$$

$$1 - k - \mathbb{E}\left[\frac{2g^{-1}(Z)}{1 + b\dot{g}^{-1}(\text{prox}_b(Z_* + \frac{1+\alpha}{2}b))}\right] = 0$$

$$\sigma^2 - \frac{b^2}{k^2}\mathbb{E}\left[2g^{-1}(Z)\left\{\frac{1+\alpha}{2} - g^{-1}\left(Z_* + \frac{1+\alpha}{2}b\right)\right\}\right] = 0,$$

where $Z \sim N(0, \gamma^2)$, $Z_* = \mu Z + \sqrt{k}\sigma G$ with $G \sim N(0,1)$, $G \perp\!\!\!\perp Z$, and $\text{prox}_b(x)$ is called proximal operator which is defined as follows:

$$\text{prox}_b(x) = \underset{u}{\text{argmin}}\left\{b\,\ln(1+\exp(u)) + \frac{1}{2}(x-u)^2\right\}.$$

The state evolution system turns out to be composed of three expected values of complicated transformations of bivariate random variables. As already mentioned, we are interested in approximating this system and then solving it. For this reason, the next two subsections deal respectively with approximating each component of the system and then solving it. A helpful reference for understanding the derivation of this specific state evolution system is (Feng and Samsworth, 2022). In particular, a recommended approach to studying the theoretical foundations of this system in that article is as follows: first, reading pages 28-29 about GAMP for GLM, then from page 33 to 35 about GAMP for linear models and convex optimization, then pages 44-47 about GAMP applied to logistic regression, and lastly the supplementary material of (Sterzinger and Kosmidis, 2023) especially pages 56-57.

3.1 State evolution system approximation

We observe that each component of the system corresponds to the expected value of a transformation of a random variable set to be equal to a certain quantity. For continuous random variables, the expected value of a bivariate transformation is expressed as an integral with the following structure:

$$\mathbb{E}[g_j(X,Y)] = \int_{S_{X,Y}} g_j(x,y) f(x,y) dxdy, \qquad j = 1,2,3 \tag{11}$$

where $g_j(\cdot)$ is the nonlinear transformation applied to (Z, Z_*), which is different for each component. $f(z, z_*)$ is the probability density function of the bivariate random variable we are considering, and the last element S_{Z,Z_*} is the support of the random vector. Thus, solving the system requires either evaluating these integrals analytically or approximating them. Since the three integrals are too complex to compute analytically, we choose to approximate them numerically. In this regard, we use a deterministic approach called Gaussian-Hermite cubature, which is essentially a multivariate extension of the homonymous quadrature method employed for univariate integral approximation. The idea of the method is to rewrite the integral in such a way that it can be approximated with a linear combination of weights and function values.

$$\int_{S_{X,Y}} h(x,y) e^{-x^2-y^2} dxdy \approx \sum_{i=1}^{N}\sum_{j=1}^{N} w_i w_j h(x_i, x_j) \tag{12}$$

In (12), the node pairs (x_i, x_j) are obtained by considering the Cartesian product of the N roots of the physicist's Hermite polynomial of order N, $H_N(x)$, with themselves. The physicist's Hermite polynomials of order N has the following structure:

$$H_N(x) = (-1)^N e^{x^2} \frac{d^N}{dx^N} e^{-x^2}.$$

On the other side, the weight pairs (w_i, w_j) are defined as the Gauss-Hermite weights associated with each node pair of order N; the following is the detailed definition:

$$w_i = \frac{2^{N-1} N! \sqrt{\pi}}{N^2 [H_{N-1}(x_i)]^2}$$

Consequently, the problem of approximating the state evolution system boils down to rewriting each integral $\mathbb{E}[g_j(Z, Z_*)]$ with $j = 1, 2, 3$ in the same form as the left-hand side of (12) by performing changes of variables and appropriate adjustments. In this regard, it turns out to be unnecessarily more complicated to consider (Z, Z_*) as the core random variable on which g_j are applied. This is because the complexity of the covariance matrix of (Z, Z_*) will lead to a cumbersome probability density function inside each expected value. Therefore, rather than approximating $\mathbb{E}[g_j(Z, Z_*)]$ directly, we rewrite it using the transformation $Z_* = T(Z, G)$. This shifts the complexity into the function g_j while keeping the probability density function f as simple as possible. This choice will help with future changes of variables. Additionally, it is worth observing that applying the following transformation:

$$\begin{bmatrix} Z \\ Z_* \end{bmatrix} = \vec{T}(Z, G) = \begin{cases} Z \\ \mu Z + \sqrt{k}\sigma G \end{cases}$$

to the integral in (11) boils down to a deterministic change of variables without the Jacobian J of $\vec{T}$. This results from the fact that the determinant of J cancels with its inverse, which is obtained from the pdf of (Z, Z_*). expressed as a function of (Z, G). The following holds:

$$\begin{aligned}
\mathbb{E}[g_j(Z, Z_*)] &= \int_{S_{Z,Z_*}} g_j(z, z_*) f(z, z_*) dz dz_* \\
&= \int_{S_{Z,G}} g_j(\vec{T}(z,g)) f_{Z,G}(\vec{T}^{-1}(z, z_*)) \cdot |\det(J_{\vec{T}^{-1}})| \cdot |\det(J_{\vec{T}})| dz dg \\
&= \int_{S_{Z,G}} s_j(z,g) f_{Z,G}(z,g) dz dg \\
&= \int_{S_{Z,G}} s_j(z,g) f_Z(z) f_G(g) dz dg.
\end{aligned}$$

In the previous expression, $s_j(\cdot)$ denotes the function $g_j(\cdot)$ rewritten in terms of Z and G, while the remaining pdf is much simpler, as it factorizes into two univariate normal distributions with mean zero and variances γ^2 and 1, respectively. By expanding each normal pdf we obtain:

$$\begin{aligned}\mathbb{E}[g_j(Z, Z_*)] &= \int_{S_{Z,G}} s_j(z, g) \frac{1}{\sqrt{2\pi\gamma^2}} \cdot e^{-\frac{z^2}{2\gamma^2}} \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{g^2}{2}} \, dzdg \\ &= \int_{S_{Z,G}} \frac{1}{2\pi\gamma} s_j(z, g) \cdot e^{-\frac{z^2}{2\gamma^2} - \frac{g^2}{2}} \, dzdg.\end{aligned}$$

Since we want to recover the structure of the integral on the left-hand side of (12), we need to perform an additional change of variables, namely:

$$\begin{bmatrix} Z \\ G \end{bmatrix} = \vec{U}(X, Y) = \begin{cases} \sqrt{2}\gamma X \\ \sqrt{2}Y \end{cases} \qquad \text{with} \quad J_{\vec{U}} = \begin{bmatrix} \sqrt{2}\gamma & 0 \\ 0 & \sqrt{2} \end{bmatrix}.$$

With this last change of variables in place, we can integrate by substitution as follows:

$$\begin{aligned}\mathbb{E}[g_j(Z, Z_*)] &= \int_{S_{X,Y}} \frac{2\gamma}{2\pi\gamma} s_j(\vec{U}(x, y)) \cdot e^{-x^2 - y^2} dxdy \\ &= \int_{S_{X,Y}} h_j(x, y) \cdot e^{-x^2 - y^2} dxdy.\end{aligned}$$

At this point, after having defined $h_j(x, y)$ (with $j = 1, 2, 3$) according to the arguments of the expected values in the state evolution system, we can apply the Gauss-Hermite cubature detailed in (12). The explicit forms of the three functions to be evaluated on the 2D grid of Gauss-Hermite nodes are given below:

$$h_1(x, y) = \frac{1}{\pi} 2g^{-1}(\sqrt{2}\gamma X)\sqrt{2}\gamma X \left\{ \frac{1+\alpha}{2} - g^{-1}\left(\text{prox}_b\left(\mu(\sqrt{2}\gamma X) + \sqrt{k}\sigma\sqrt{2}Y + \frac{1+\alpha}{2} b \right) \right) \right\}$$

$$h_2(x, y) = \frac{1}{\pi} \frac{2g^{-1}(\sqrt{2}\gamma X)}{1 + b\dot{g}^{-1}(\text{prox}_b(\mu(\sqrt{2}\gamma X) + \sqrt{k}\sigma\sqrt{2}Y + \frac{1+\alpha}{2} b))}$$

$$h_3(x, y) = 2g^{-1}(\sqrt{2}\gamma X) \left\{ \frac{1+\alpha}{2} - g^{-1}\left(\mu(\sqrt{2}\gamma X) + \sqrt{k}\sigma\sqrt{2}Y + \frac{1+\alpha}{2} b \right) \right\}.$$

Unlike other approximation methods, the Gauss-Hermite cubature guarantees uniform accuracy over all dimensions of integration; moreover, by choosing N as the number of nodes, one can arbitrarily calibrate the precision of the approximation. The function in `brglm2` that approximates, via Gauss-Hermite cubature, the state evolution system, when an intercept is not included, is `se0()`. This function, with the support of Prof. Ioannis Kosmidis, has been vectorized in order to reduce the elapsed time used to approximate the system.

3.2 Case with intercept

This subsection deals with the theoretical background behind `se1()`, which is the function that approximates the state evolution system when an intercept θ_0 is included in the logistic regression model. In this case the GLM we are considering is defined as follows:

$$y_i \sim \text{Ber}(\mu_i) \quad \mathbb{P}(y_i = 1 | \mathbf{x}_i^\top) = g^{-1}(\theta_0 + \mathbf{x}_i^\top \boldsymbol{\beta}), \quad \text{with} \quad g^{-1}(\cdot) = \frac{\exp(\cdot)}{1 + \exp(\cdot)} \quad \text{and} \quad i = 1, \ldots, n$$

The associated GAMP recursion determines a state evolution system of four nonlinear equations in four unknowns (μ, b, σ, ι) with the following structure:

$$0 = \mathbb{E}[g^{-1}(Z_1)Z_1Q_+(\alpha, b, Z_2)] - \mathbb{E}[g^{-1}(-Z_1)Z_1Q_-(\alpha, b, Z_2)]$$

$$1 - k = \mathbb{E}\left[\frac{g^{-1}(Z_1)}{1 + b\dot{g}^{-1}(\text{prox}_b(\frac{1+\alpha}{2}b + Z_2))}\right] + \mathbb{E}\left[\frac{g^{-1}(-Z_1)}{1 + b\dot{g}^{-1}(\text{prox}_b(\frac{1+\alpha}{2}b - Z_2))}\right]$$

$$\frac{\sigma^2k^2}{b^2} = \mathbb{E}[g^{-1}(Z_1)Q_+(\alpha, b, Z_2)^2] + \mathbb{E}[g^{-1}(-Z_1)Q_-(\alpha, b, Z_2)^2]$$

$$0 = \mathbb{E}[g^{-1}(Z_1)Q_+(\alpha, b, Z_2)] - \mathbb{E}[g^{-1}(-Z_1)Q_-(\alpha, b, Z_2)],$$

where

$$Q_+(\alpha, b, Z) = \frac{1+\alpha}{2} - g^{-1}\left(\text{prox}_b\left(\frac{1+\alpha}{2}b + Z\right)\right), \quad Q_-(\alpha, b, Z) = \frac{1+\alpha}{2} - g^{-1}\left(\text{prox}_b\left(\frac{1+\alpha}{2}b - Z\right)\right),$$

and $Z_1 = \gamma Z + \theta_0$, $Z_2 = \mu\gamma Z + \sqrt{k}\sigma G + \iota$, $Z, G \sim N(0, 1)$, with $Z \perp\!\!\!\perp G$. Lastly, ι represents the limit of the DY estimate of θ_0 as $n \longrightarrow \infty$. As in the case without intercept, the solution $(\mu_*, b_*, \sigma_*, \iota_*)$ of this state evolution system governs the aggregate proportional asymptotic behavior of the DY estimator when an intercept is included.

3.3 System approximation with intercept

A very similar approach to the case without an intercept has been adopted. Naturally, a few adjustments are required to rewrite each component of this new system before applying Gauss–Hermite cubature. In this case, the first change of variables (for which the determinant of the Jacobian can be ignored) is done through the following transformation:

$$\begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix} = \vec{T}(Z, G) = \begin{cases} \gamma Z + \theta_0 \\ \mu\gamma Z + \sqrt{k}\sigma G + \iota \end{cases} \tag{13}$$

Consequently, we can rewrite each expected value with $j = 1, 2, 3, 4$ as:

$$\begin{aligned} \mathbb{E}[g_j(Z_1, Z_2)] &= \int_{S_{Z_1,Z_2}} g_j(z_1, z_2) f_{Z_1,Z_2}(z_1, z_2) dz_1 dz_2 \\ &= \int_{S_{Z,G}} g_j(\vec{T}(z, g)) f_{Z,G}(\vec{T}^{-1}(z_1, z_2)) dz dz g \\ &= \int_{S_{Z,G}} s_j(z, g) f_Z(z) f_G(g) dz dz g \\ &= \int_{S_{Z,G}} s_j(z, g) \frac{1}{2\pi} e^{-\frac{z^2}{2} - \frac{g^2}{2}} dz dz g. \end{aligned}$$

Since, as before, we want the integrals to be expressed in the same form as the left-hand side of (12), we have to perform another change of variables:

$$\begin{bmatrix} Z \\ G \end{bmatrix} = \vec{U}(X, Y) = \begin{cases} \sqrt{2}X \\ \sqrt{2}Y \end{cases} \qquad \text{with} \quad J_{\vec{U}} = \begin{bmatrix} \sqrt{2} & 0 \\ 0 & \sqrt{2} \end{bmatrix}. \tag{14}$$

As a result, the final j^{th} integral will be:

$$\mathbb{E}[g_j(Z_1, Z_2)] = \int_{S_{X,Y}} \frac{1}{2\pi} s_j(\vec{U}(x, y)) e^{-x^2-y^2} dx dy = \int_{S_{X,Y}} h_j(x, y) e^{-x^2-y^2} dx dy.$$

Once rewritten in this way, the system is ready to be approximated via Gauss-Hermite cubature. Below are shown the four functions h_j $(j = 1, 2, 3, 4)$ to be evaluated on the 2D grid of physicist's Hermite polynomial

nodes of order N:

$$
\begin{aligned}
h_1(x,y) &= g^{-1}(\gamma\sqrt{2}x+\theta_0)(\gamma\sqrt{2}x+\theta_0)Q_+(\alpha,b,\sqrt{2}x+\sqrt{k}\sigma\sqrt{2}y+\iota)\\
&\quad - g^{-1}(-\gamma\sqrt{2}x-\theta_0)(\gamma\sqrt{2}x+\theta_0)Q_-(\alpha,b,\sqrt{2}x+\sqrt{k}\sigma\sqrt{2}y+\iota)
\end{aligned}
$$

$$
\begin{aligned}
h_2(x,y) &= \frac{g^{-1}(\gamma\sqrt{2}x+\theta_0)}{1+b\dot{g}^{-1}(\mathrm{prox}_b(\frac{1+\alpha}{2}b+\mu\gamma\sqrt{2}x+\sqrt{k}\sigma\sqrt{2}y+\iota))}\\
&\quad + \frac{g^{-1}(-\gamma\sqrt{2}x-\theta_0)}{1+b\dot{g}^{-1}(\mathrm{prox}_b(\frac{1+\alpha}{2}b-\mu\gamma\sqrt{2}x-\sqrt{k}\sigma\sqrt{2}y-\iota))}
\end{aligned}
$$

$$
\begin{aligned}
h_3(x,y) &= g^{-1}(\gamma\sqrt{2}x+\theta_0)Q_+(\alpha,b,\mu\gamma\sqrt{2}x+\sqrt{k}\sigma\sqrt{2}y+\iota)^2\\
&\quad + g^{-1}(-\gamma\sqrt{2}x-\theta_0)Q_-(\alpha,b,\mu\gamma\sqrt{2}x+\sqrt{k}\sigma\sqrt{2}y+\iota)^2
\end{aligned}
$$

$$
\begin{aligned}
h_4(x,y) &= g^{-1}(\gamma\sqrt{2}x+\theta_0)Q_+(\alpha,b,\mu\gamma\sqrt{2}x+\sqrt{k}\sigma\sqrt{2}y+\iota)\\
&\quad - g^{-1}(-\gamma\sqrt{2}x-\theta_0)Q_-(\alpha,b,\mu\gamma\sqrt{2}x+\sqrt{k}\sigma\sqrt{2}y+\iota)
\end{aligned}
$$

As already mentioned, by performing the change of variables in (13), we now treat (Z, G) as the core random variables instead of (Z, Z_*). This choice shifts all the complexity into each function $h_j(x, y)$, making the subsequent change of variables in the PDF (14) (required for applying Gauss–Hermite cubature) much simpler. To summarize, the functions `se0()` and `se1()` approximate the mappings $\vec{\mathrm{F}}_0(\mu, b, \sigma) : \mathbb{R}^3 \to \mathbb{R}^3$ and $\vec{\mathrm{F}}_1(\mu, b, \sigma, \iota) : \mathbb{R}^4 \to \mathbb{R}^4$, respectively. When passed to a solver, these functions are evaluated iteratively according to the solver's update rules, allowing the search for their roots. The structure of the solver function `solve_se()` is detailed in the next section. In `brglm2` more flexibility is provided, allowing the user to solve the state evolution system in the case γ, , θ_0 are unknown; we will ignore this extension since it hasn't been the focus of the project. For this reason, from then on, we will always assume to know the oracle values of all three parameters (γ, k, θ_0) needed to solve the system.

3.4 State evolution system solver

This section deals with a few details related to the `solve_se()` function. This solver is just a wrapper of the function `nleqslv()`, from the homonymous package. Once `se0()` or `se1()` are supplied to the solver, `solve_se()` attempts to compute their numerical root using either the Newton or the Broyden method (default is Broyden double dog leg strategy). In order to guarantee robustness, the user can also specify the first n iterations to be computed using the function `optim()` (default is Nelder-Mead method). In the context of trust region algorithms, Broyden's method works as a multivariate Newton-Raphson, where at each iteration the Jacobian is approximated and inverted using a specific updating formula. Moreover, depending on the global strategy used, we have some flexibility in the proposals used to solve the trust region subproblem at each iteration. The following sections describe the default setting in `nleqslv()`.

3.5 Broyden's method

Starting from the core default solver in `nleqslv()`, this section deals with Broyden's algorithm. The function supplied to this method is either `se0()` or `se1()`; in this regard, we are interested in finding the root of these functions, which means solving the nonlinear systems they determine (either in the case with an intercept or not). If we include θ_0 and define $\mathrm{F}(\mathbf{x}) = \mathrm{F}(\mu, b, \sigma, \iota) = \mathrm{se0}(\mu, b, \sigma, \iota)$, the Broyden's method is defined as follows:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{B}_k^{-1}(\mathbf{x}_k)\mathbf{F}(\mathbf{x}_k),$$

where $\mathbf{B}_k$ is the approximation at iteration k of the Jacobian $\mathbf{J}_k$ of `se1()`. Given $\Delta\mathbf{x}_{k+1} = \mathbf{x}_{k+1} - \mathbf{x}_k$ and $\Delta\mathbf{F}(x_{k+1}) = \mathrm{se1}(\mathbf{x}_{k+1}) - \mathrm{se1}(\mathbf{x}_k)$, the updating formula for the Jacobian is defined as follows:

$$\mathbf{B}_{k+1} = \mathbf{B}_k + \frac{\Delta\mathbf{F}(x_{k+1}) - \mathbf{B}_k\Delta\mathbf{x}_{k+1}}{\|\Delta\mathbf{x}_{k+1}\|_2^2}\Delta\mathbf{x}_{k+1}^\top. \tag{15}$$

The updating mechanism works by adding to the approximated Jacobian at step k, $\mathbf{B}_k$, a correction whose numerator $\Delta\mathbf{F}(x_{k+1}) - \mathbf{B}_k\Delta\mathbf{x}_{k+1}$ represents the error in the function value committed by using $\mathbf{B}_k$ instead of the actual Jacobian $\mathbf{J}_k$; this become straightforward if we think about the Jacobian as a multivariate generalization of the first derivative of a vector function and then approximate it with its incremental ratio. Lastly, the remaining part in (15) is just a unit vector ensuring that the direction of the correction is the same as that of the update. The previously introduced Broyden's method is applied within the framework of trust region algorithms, namely, optimization methods that are considered reliable only over a region around the current iteration. At each step, given $\mathbf{m}_k(\mu_k, b_k, \sigma_k, \iota_k)$ which is the approximation of `se1()` using Broyden double dog leg, depending on the ratio:

$$\rho_{k+1} = \frac{\|\text{se1}(\mu_{k+1}, b_{k+1}, \sigma_{k+1}, \iota_{k+1})\| - \|\text{se1}(\mu_k, b_k, \sigma_k, \iota_k)\|}{\|\mathbf{m}(\mu_{k+1}, b_{k+1}, \sigma_{k+1}, \iota_{k+1})\| - \|\mathbf{m}(\mu_k, b_k, \sigma_k, \iota_k)\|} \tag{16}$$

the trust region is either expanded ($\rho \approx 1$) or shrunk ($\rho \approx 0$). The region is contracted when the function approximation performs poorly in finding the root, ensuring safer and smaller iterations. Conversely, if the approximation performs well, the region is expanded, giving the algorithm greater flexibility to explore the search space. The Double Dogleg global strategy determines how the step is proposed within the trust region. First, a gradient descent step is computed. From this new point, a Broyden step is generated. If the resulting step lies within the trust region, it is accepted. Otherwise, a step is chosen to be on the boundary of the trust region and along the line connecting the gradient descent and Broyden steps.

3.6 Comparison Julia vs R code

The functions developed during this project were partially inspired by existing Julia code, detailed in the repository https://github.com/psterzinger/MDYPL. In this project, we aimed to create a simplified R version of the Julia functions `solve_mDYPL_SE()` and `mDYPL_SE()`. The idea was to reduce the flexibility of the solver and the approximation function in order to obtain a gain in terms of elapsed time.

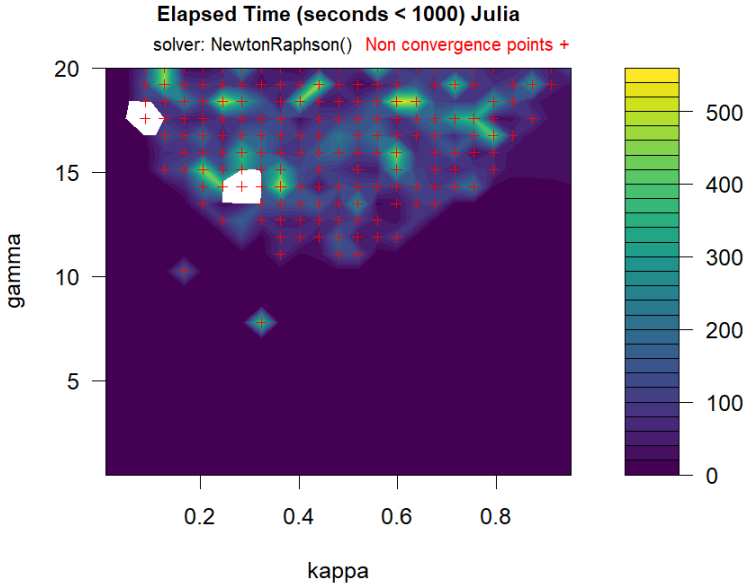

Figure 2: Contour plot of the elapsed time (in seconds) for `solve_mDYPL_SE()` across different combinations of (k, γ) values. The state evolution system is approximated using the global adaptive subdivision strategy `Cuba.cuhre()`, and solved with the `NewtonRaphson()` method from `NonlinearSolve()`. Red crosses indicate points where the algorithm failed to converge, while white regions correspond to (k, γ) combinations that were excluded because their elapsed times were disproportionately large, which would have obscured the intermediate variations in runtime.

The primary differences between the R and Julia implementations lie in both the solver and the approximation method used for the state evolution system. In Julia, `mDYPL_SE()` approximates each component of the state evolution equations using a global adaptive subdivision strategy, implemented via the `cuhre()` function from the Cuba library https://cubajl.readthedocs.io/en/v0.3.1/.

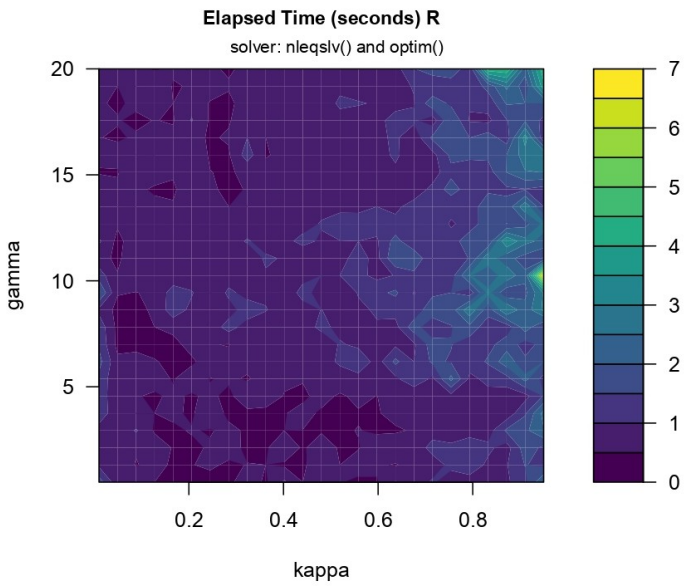

Figure 3: Contour plot of the elapsed times (in seconds) for `solve_se()` across different combinations of (k, γ) values. The state evolution system is approximated using Gauss-Hermite cubature, solved with the Broyden double-dogleg method, and initialized using the Nelder-Mead algorithm for the first iterations.

By contrast, the R function `solve_se()` employs the Gauss-Hermite cubature method, as detailed in this report. A second key difference concerns the nonlinear solver: the Julia implementation uses either `TrustRegion()` or `NewtonRaphson()` from the NonlinearSolve library https://docs.sciml.ai/NonlinearSolve/stable/, whereas the R implementation uses as default the Broyden double-dogleg method. Figure 2 and 3 show a comparison between elapsed times for the R code and the Julia code.

3.7 Solution accuracy

Since we are comparing the performance of two different solvers for the same state evolution system, namely, `solve_se()` and `solve_mDYPL_SE()`, it could be interesting to compare the accuracy of the resulting solutions.

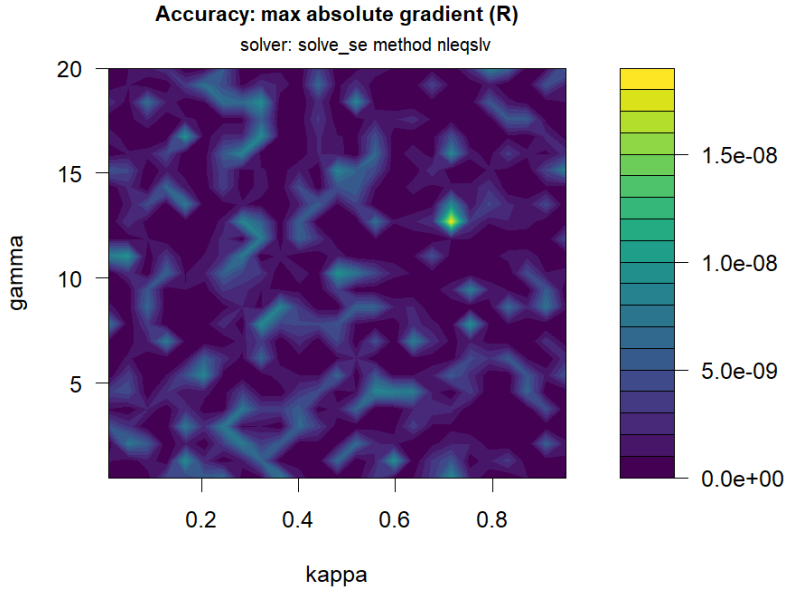

Figure 4: Contour plot of the accuracy of the solver `solve_se()` measured as in (17), across different combinations of (k, γ) values. The solver uses the approximation function se0$(\cdot)$ in R and a Broyden double dogleg algorithm to solve the root-finding problem.

This is done by evaluating se0(μ, b, σ) on the estimated solution $(\hat{\mu}_*, \hat{b}_*, \hat{\sigma}_*)$ using both `solve_se()` and

`solve_mDYPL_SE()` on a grid of (k, γ) values. Clearly, a higher accuracy means $\text{se0}(\hat{\mu}_*, \hat{b}_*, \hat{\sigma}_*) \approx \mathbf{0}$. Since $\mathbf{0}$ is a vector, we can plot for each (k, γ) the following measure of accuracy:

$$\text{acc}(k, \gamma) = \max_{i=1,2,3} |\text{se0}_i(\hat{\mu}_*, \hat{b}_*, \hat{\sigma}_*)|, \tag{17}$$

Figure 4 and 5 are two contour plots representing the overall accuracy of the R function `solve_se()` and the Julia function `solve_mDYPL_SE()` measured as in (17).

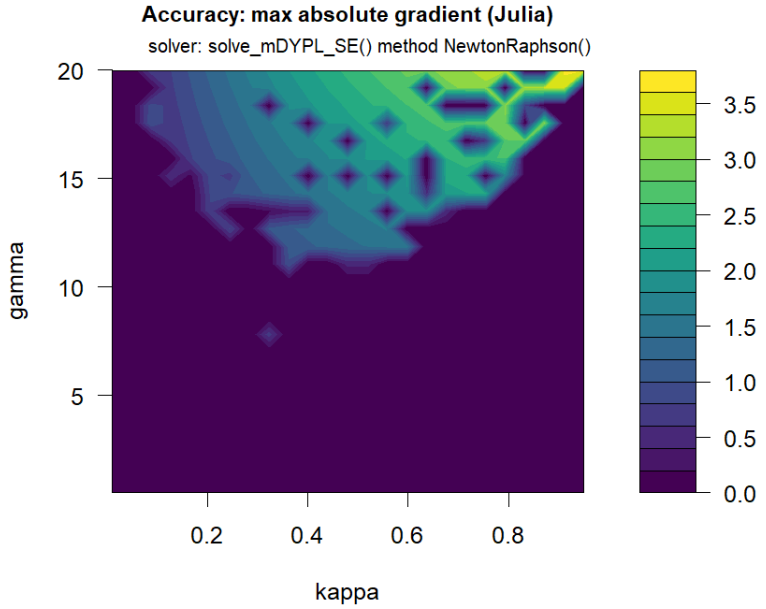

Figure 5: Contour plot of the accuracy of the solver `solve_mDYPL_SE()` measured as in (17), across different combinations of (k, γ) values. The solver uses the Julia approximation function `mDYPL_SE()` and a `NewtonRaphson()` algorithm to solve the root-finding problem.

As we can observe, the Julia solver exhibits lower accuracy than the R one throughout a broad semicircular region of the $(k,)$ plane, located in the upper part of the graph.

References

A. Albert and J.A. Anderson. On the existence of maximum likelihood estimates in logistic regression models. *Biometrika*, 1984.

Venkataraman R. Rush C. Feng, O.Y. and R.J. Samsworth. A unifying tutorial on approximate message passing. *Foundations and Trends in Machine Learning*, 2022.

P. Sterzinger and I. Kosmidis. Diaconis-ylvisaker prior penalized likelihood for $p/n \to k \in (0, 1)$ logistic regression. 2023. arXiv preprint.

P. Sur and E.J. Candes. A modern maximum-likelihood theory for high-dimensional logistic regression. *Proceedings of the National Academy of Sciences*, 2019.